

推导完全分类

我会把证明收敛到集合论形式：在 $P1 \wedge P2$ 下，子声部可开仓集合被 tree 覆盖域、BSP 域与 `hostOf` 定义域的交集所上界。关键不是买卖点是否存在，而是它能否成为树内 carrier 的合法 voice certificate。

是的，这里可以严格推出一个结论，但要把命题写得精确一点：

$$P1 \wedge P2 \implies \pi_{\Theta}^{bsp} \text{ 只能交易“可由 extract tree 精确 host 的 BSP”}。$$

所以如果子声部 BSP 主要落在 orphan frontier 上，那么：

$$\pi_{\Theta}^{bsp} \text{ 的子声部结构性为 } 0。$$

这不是买卖点本身没 alpha，而是当前 $P1 \wedge P2$ 语义下，那些买卖点不能成为合法 voice certificate。下面给出严格推导。

1. 基本集合

令第 i 根 bar 后的全量塔结构元素集合为

$$U_i = \bigsqcup_{\ell} U_{i,\ell}.$$

其中 $U_{i,\ell}$ 是第 ℓ 级全部走势元素，包括已被高一级 compose 包含的元素，也包括尚未被高一级结构吸收的 frontier 元素。

`extract_elements` 生成的树记为

$$T_i \subseteq U_i.$$

根据 P1, T_i 不是全量森林，而是：

$$T_i = \downarrow r_i$$

其中 r_i 是 tower 最高非空级别上的最高级走势根， $\downarrow r_i$ 表示从 r_i 沿 Compose.subs 向下可达的所有元素。

因此 orphan frontier 集合为：

$$F_i = U_i \setminus T_i.$$

你给的数据：

$$|T_i \cap U_{i,0}| / |U_{i,0}| = 486 / 1360 \approx 36\%$$

说明大量 L0 走势属于

$$F_i.$$

2. BSP 全格集合

令所有买卖点候选集合为

$$B_i.$$

根据 P2, BSP 来自 classification 全格:

$$B_i \subseteq \bigsqcup_{\ell} \text{BSP}(U_{i,\ell}).$$

也就是说, BSP 不只来自 T_i , 而来自整个 tower 全格 U_i 。

每个 BSP 候选 $g \in B_i$ 有:

$$\ell(g), \quad \delta(g) \in \{+1, -1\}, \quad s(g),$$

其中 $s(g)$ 是 source_index, 也就是该 BSP 对应的严格右端点。

3. hostOf 的定义域

P2 规定 hostOf 是严格右端点命中:

$$\text{host}_i(g) = c \iff c \in T_i \wedge \rho(c) = s(g).$$

若不存在这样的 $c \in T_i$, 则:

$$\text{host}_i(g) = \perp.$$

这里 $\rho(c)$ 是 tree 元素 c 的右端点。

因此 hostOf 的定义域是:

$$\text{Dom}(\text{host}_i) = \{g \in B_i : s(g) \in \rho(T_i)\}.$$

其中:

$$\rho(T_i) = \{\rho(c) : c \in T_i\}.$$

所以只要某个 BSP g 的 source_index 不在 T_i 的右端点集合中, 就必然:

$$\text{host}_i(g) = \perp.$$

这一步是 P2 的直接推论。

4. 合法买卖点证书

π_{Θ}^{bsp} 中, BSP 不能直接开仓; 它必须先成为一个合法 certificate。

定义合法证书集合:

$$\Gamma_i = \{g \in B_i : \text{host}_i(g) \neq \perp\}.$$

如果:

$$g \notin \Gamma_i,$$

则它不会生成 voice。

买卖点候选 g 开仓时生成的 position voice 是:

$$v_g = (\text{host}_i(g), g, \delta(g)).$$

所以

$$g \text{ 能开 voice} \iff \text{host}_i(g) \neq \perp.$$

这已经说明：

$$\text{BSP 全格存在} \neq \text{BSP 可交易}.$$

可交易的只是：

$$B_i \cap \text{Dom}(\text{host}_i).$$

5. 子声部 active 的必要条件

令 raw voice set 为

$$R_i = H_i^{\text{live}} \cup O_i,$$

其中：

$$O_i = \{v_g : g \in \Gamma_i^{\text{acc}}\}$$

是被解释器接受的新开仓 voice,

$$H_i^{\text{live}}$$

是上一时刻留下且未关闭的 held voices。

最终活动声部为：

$$A_i = \text{AncOK}(R_i) = \{v \in R_i : \text{Anc}(v) \subseteq R_i\}.$$

于是：

$$v \in A_i \iff v \in R_i \wedge \text{Anc}(v) \subseteq R_i.$$

若 v 是 $\text{depth} > 0$ 子声部，则存在父声部：

$$p(v).$$

祖先闭合要求：

$$v \in A_i \implies p(v) \in A_i.$$

更强地：

$$v \in A_i \implies p(v) \in R_i.$$

也就是说，父声部必须已经是 raw active voice；AncOK 不会自动生成父声部。

6. AncOK 是过滤器，不是生成器

严格地：

$$\text{AncOK}(R_i) \subseteq R_i.$$

所以它永远不会添加节点。

若子声部 $u \in R_i$ ，但父声部 $p(u) \notin R_i$ ，则：

$$u \notin \text{AncOK}(R_i).$$

这说明：

$$a_{u,i} \leq a_{p(u),i}$$

是约束，不是生成规则。

它表示：

$$a_{p(u),i} = 0 \Rightarrow a_{u,i} = 0.$$

不是：

$$a_{u,i} = 1 \Rightarrow \text{系统自动生成 } a_{p(u),i} = 1.$$

如果使用的是“子声部要开，所以强制生成父声部”，那对应的是另一个算子：

$$\text{UpClose}(R_i) = R_i \cup \text{Anc}(R_i),$$

而不是：

$$\text{AncOK}(R_i).$$

两者相反：

$$\text{AncOK}(R_i) \subseteq R_i \subseteq \text{UpClose}(R_i).$$

因此，§9.1 的 AncOK 语义下，子声部不能反向制造父声部。

7. 子声部结构性为零的定理

定义可开子声部 BSP 集合：

$$B_i^{sub,open} = \{g \in B_i : \text{host}_i(g) \neq \perp, v_g \text{ 是 depth} > 0 \text{ voice, } \text{Anc}(v_g) \subseteq R_i\}.$$

则：

$$A_i^{sub} = \{v \in A_i : d(v) > 0\} \subseteq \{v_g : g \in B_i^{sub,open}\}.$$

因此：

$$B_i^{sub,open} = \emptyset \implies A_i^{sub} = \emptyset.$$

证明

取任意

$$v \in A_i^{sub}.$$

因为

$$A_i = \text{AncOK}(R_i),$$

所以：

$$v \in R_i.$$

若 v 是新开子声部，则

$$v = v_g$$

且

$$g \in \Gamma_i^{acc}.$$

因此：

$$\text{host}_i(g) \neq \perp.$$

又因为 $v \in A_i$ ，所以：

$$\text{Anc}(v) \subseteq R_i.$$

因此：

$$g \in B_i^{sub,open}.$$

所以：

$$A_i^{sub} \subseteq \{v_g : g \in B_i^{sub,open}\}.$$

若右侧为空，则左侧为空。

证毕。

8. P1∧P2 如何推出 orphan frontier BSP 不可交易

令某个 BSP g 来自 orphan frontier 元素 $e \in F_i$ ，且：

$$s(g) = \rho(e).$$

若：

$$\rho(e) \notin \rho(T_i),$$

则由 P2：

$$\text{host}_i(g) = \perp.$$

于是：

$$g \notin \Gamma_i.$$

因此它不会生成：

$$v_g.$$

所以：

$$e \in F_i \wedge \rho(e) \notin \rho(T_i) \implies g \text{ 不可交易。}$$

这说明：

全格 BSP 中落在 orphan frontier 且端点不被 T_i 命中的信号，在 π_{Θ}^{bsp} 中不可见。

9. 结构性恒零的充要条件

在当前 $P1 \wedge P2$ 下, π_{Θ}^{bsp} 的子声部为零的精确条件是:

$$\boxed{\forall i, \quad B_i^{sub,open} = \emptyset.}$$

展开就是:

$$\boxed{\forall i, \nexists g \in B_i \text{ such that } \begin{cases} s(g) \in \rho(T_i), \\ v_g \text{ 是 depth} > 0 \text{ voice,} \\ \text{Anc}(v_g) \subseteq R_i, \\ g \text{ 被解释器接受.} \end{cases}}$$

若此条件成立, 则:

$$\boxed{\forall i, \quad A_i^{sub} = \emptyset.}$$

这就是“子声部结构性恒 = 0”的严格数学版本。

注意: P1 和 P2 本身推出的是上界:

$$A_i^{sub} \subseteq \text{hostable BSP on } T_i.$$

要推出恒等于零, 还需要事实:

hostable 子声部 BSP + active ancestor chain 为空。

你的数据里, “BSP 全格大量在 orphan frontier, 而 tree 只覆盖 36% L0”, 正是在实证上验证这个条件。

10. 三方向不可同时取

现在定义三个要求。

P1

$$T_i = \downarrow r_i,$$

只取最高 compose chain。

P2

$$\text{host}_i(g) = c \iff c \in T_i \wedge \rho(c) = s(g).$$

严格右端点命中。

D

所有全格 BSP, 至少所有合法子声部 BSP, 都应能进入 π^{bsp} 交易解释器。

形式化为:

$$\boxed{\forall g \in B_i^{sub,relevant}, \quad \text{host}_i(g) \neq \perp.}$$

若存在 orphan frontier BSP:

$$g \in B_i^{sub,relevant},$$

其 source index 满足：

$$s(g) \notin \rho(T_i),$$

则由 P2：

$$\text{host}_i(g) = \perp.$$

这与 D 矛盾。

因此：

$\boxed{P1 \wedge P2 \wedge D}$

在存在 orphan frontier BSP 的情况下不可同时成立。

所以只有三条路。

路线 A：打破 P1

把 extract tree 从：

$$T_i = \downarrow r_i$$

改成全量森林或至少包含 frontier：

$$T'_i = T_i \cup F_i^{\text{needed}}.$$

这样 orphan BSP 可能有 host。

代价：

改变 extract_elements 的树宇宙，复活 P1 要避免的问题。

也就是你说的“547 复活”风险。

路线 B：打破 P2

把 hostOf 改成区间包含或最近容器：

$$\text{host}'(g) = \min\{c \in T_i : s(g) \in I(c)\}$$

或其他 containment rule。

这样 orphan frontier BSP 可以挂到某个高层容器。

代价：

host 不再是 strict right endpoint hit，复活 P2 要避免的问题。

也就是你说的“638 复活”风险。

路线 C：保留 $P1 \wedge P2$

接受：

π_{Θ}^{bsp} 只交易 hostable BSP，不交易全格 BSP。

于是 orphan frontier 上的 BSP 对子声部没有作用。

若子声部信号主要落在 orphan frontier，则：

$$\pi_{\Theta}^{bsp} \text{ 子声部结构性为 } 0.$$

这不是 workaround，而是 $P1 \wedge P2$ 的逻辑后果。

11. 为什么 workaround 被禁止是对的

11.1 伪造父链

若

$$\text{host}(g) = \perp$$

却人为创建：

$$\text{parent}(g) = \text{some container},$$

则你已经放弃 P2 的严格 hostOf。

这是路线 B 的伪装版本。

11.2 用级别差当父子

如果只因：

$$\ell(\text{parent}) = \ell(\text{child}) + 1$$

就建立父子关系，而不要求 Compose.subs 或 strict host，则你已经放弃 P1 的树关系。

这是路线 A 的伪装版本。

11.3 放宽 hostOf 到区间包含

这直接是打破 P2。

因此这些 workaround 都不是解决矛盾，而是绕开 P1 或 P2。

12. 为什么这说明 goal #5 是 wrong object

若目标 #5 是：

$$\text{在当前 } P1 \wedge P2 \text{ 语义下，用全格 BSP 触发多重赋格子声部 } \alpha$$

那么它要求 D 成立。

但 $P1 \wedge P2$ 排除了大量全格 BSP，尤其是 orphan frontier BSP。

所以 goal #5 实际测试到的是：

$$\text{最高 compose chain 可达树上的 hostable BSP 子声部}$$

而不是：

classification 全格 BSP 子声部

若子声部机会主要在 orphan frontier，则当前 π^{bsp} 子声部为零是结构必然。

所以你说的方向 C：

保留 $P1 \wedge P2$ ，则接受定理。

是数学上最干净的解释。

13. 对 ΔSharpe 的严格推论

若：

$$A_i^{sub} = \emptyset \quad \forall i,$$

则 π_{Θ}^{bsp} 的子声部项恒为零。

令 baseline 策略为不含子声部的策略：

$$\pi_{\Theta}^{base}.$$

若两者唯一差异就是子声部项，则：

$$\pi_{\Theta}^{bsp}(x_i) = \pi_{\Theta}^{base}(x_i) \quad \forall i.$$

因此净头寸相同：

$$N_i^{bsp} = N_i^{base} \quad \forall i.$$

NAV 递推：

$$W_{i+1} = W_i + N_i \Delta P_{i+1} - C_i.$$

若成本模型也相同，则：

$$W_i^{bsp} = W_i^{base} \quad \forall i.$$

因此收益序列相同：

$$r_i^{bsp} = r_i^{base} \quad \forall i.$$

所以：

$$\Delta\text{Sharpe} = 0.$$

这不是 alpha 被市场否定，而是子声部没有进入目标头寸。

14. 这否证了什么，没否证什么

当前定理否证的是：

在 $P1 \wedge P2$ 的 host 语义下，全格 BSP 能驱动子声部。

它没有否证：

买卖点择时本身有没有 α 。

因为全格 BSP 中大量 orphan frontier 信号并没有进入 π^{bsp} 。

所以更精确地说：

$$\Delta \text{Sharpe} = 0$$

在这里证明的是：

当前对象没有被实际交易到，或只交易了 hostable 子集。

不是：

所有买卖点择时无效。

15. 最终定理

可以把整个问题压缩成如下定理。

定理

设：

1. $T_i = \downarrow r_i$ ，即 extract tree 只含最高 compose chain 可达元素；
2. BSP 来自全塔 U_i ；
3. hostOf 为 strict endpoint hit：

$$\text{host}_i(g) = c \iff c \in T_i \wedge \rho(c) = s(g);$$

4. voice 只能由 hosted certificate 生成；
5. AncOK 是过滤器：

$$A_i = \{v \in R_i : \text{Anc}(v) \subseteq R_i\};$$

6. 存在大量子声部相关 BSP g 位于 orphan frontier，且

$$s(g) \notin \rho(T_i).$$

则这些 BSP 不能生成 voice。若所有子声部相关 BSP 均如此，或其 hostable 子集没有 active ancestor chain，则：

$$A_i^{sub} = \emptyset \quad \forall i.$$

因此：

$$\pi_{\Theta}^{bsp} \text{ 的子声部项恒为 } 0.$$

若 #5 的边际贡献只来自这些子声部，则：

$$\Delta \text{Sharpe} = 0.$$

证明

由 3 和 6：

$$s(g) \notin \rho(T_i) \implies \text{host}_i(g) = \perp.$$

由 4：

$$\text{host}_i(g) = \perp \implies g \notin \Gamma_i \implies g \text{ 不生成 } v_g.$$

由 5，任何 $\text{depth} > 0$ voice 若要 active，其自身和全部祖先必须已经在 raw active set 中。

若所有子声部 BSP 都不能生成 voice，或没有完整 active ancestor chain，则：

$$A_i^{sub} = \varnothing.$$

若子声部项恒为空，则策略等于不含子声部的 baseline，故净头寸、NAV、收益序列、Sharpe 全部相同。

证毕。

最终结论

$P1 \wedge P2$

把 π_{Θ}^{bsp} 的可交易信号域从“classification 全格 BSP”收缩成：

$B_i^{hosted} = \{g \in B_i : s(g) \in \rho(T_i)\}.$

而 P1 的 tree 只覆盖最高 compose chain，因此大量 orphan frontier BSP 不在可交易域内。

所以：

π_{Θ}^{bsp} 子声部恒零不是买卖点无 alpha 的证明，

而是：

$P1 \wedge P2$ 下，子声部信号无法合法 host 的结构定理。

若要让全格 BSP 参与子声部，只能三选一：

方案	代价
A : 扩大 extract tree 到全森林/frontier	打破 P1
B : 放宽 hostOf 到区间包含/最近容器	打破 P2
C : 保留 P1^P2	接受子声部只交易 hostable 子集，甚至结构性为

所以如果你坚持 P1 和 P2 都正确，那么 C 是唯一自洽结论：

`\boxed{\text{当前 goal #5 的多声部 BSP 对冲层，在这个对象定义下是 wrong object。}}`



我会先证明保留 P1/P2 与让全格买卖点都可交易三者不相容，然后给出对齐信号域与 carrier 域的重构方案。接着证明重构后的策略仍全定义、互斥，并且子声部不再结构性为零。

可以解决，但不能在原来的三个约束同时保持不变的情况下解决。严格地说，你现在面对的是一个定义域冲突，不是参数、权重或 alpha 问题。

核心结论是：

$P1_{\text{extract tree only}} + P2_{\text{hostOf 只在 tree 内严格右端点命中}} + D_{\text{全格 BSP 都可作为 } \pi^{\text{bsp}} \text{ 操作证书}}$ 三者不可同时成立

所以解决方案必须是三选一：

方案	代价
A	扩大操作 host 的载体宇宙：不再只用最高 compose tree
B	放宽 hostOf：不再只做严格右端点命中
C	保留 P1/P2：接受 orphan frontier BSP 不参与子声部

最干净的解决方案是 A 的弱化版：

保留 T_i = 最高 compose tree 作为结构视图，但另建一个 endpoint-complete 的操作 carrier

也就是：

结构视图 $T_i \neq$ 操作 host 视图 K_i .

这样可以同时保留：

- `extract_elements` 的 bit-exact tree 语义；
- hostOf 的严格右端点命中；
- 买卖点在全格上的操作可用性。

但必须承认：如果 P2 被定义成“hostOf 必须命中 T_i 里的元素”，那就无解；只有把 P2 改成“hostOf 必须在操作 carrier forest K_i 中严格右端点命中”，才可解。

1. 原约束的不可能定理

设全量 tower 元素集合为：

$$U_i = \bigsqcup_{\ell} U_{i,\ell}.$$

`extract_elements` 生成的树为：

$$T_i = \downarrow r_i,$$

其中 r_i 是最高非空级别走势根， $\downarrow r_i$ 表示沿 `compose.subs` 可达的所有元素。

所以 orphan frontier 为：

$$F_i = U_i \setminus T_i.$$

BSP 全格集合为：

$$B_i.$$

每个 BSP $g \in B_i$ 有 source index:

$$s(g).$$

原 P2 的 hostOf 是:

$$\text{host}_i^T(g) = c \iff c \in T_i \wedge \rho(c) = s(g).$$

若不存在, 则:

$$\text{host}_i^T(g) = \perp.$$

因此:

$$\text{Dom}(\text{host}_i^T) = \{g \in B_i : s(g) \in \rho(T_i)\}.$$

如果 g 来自 orphan frontier, 并且:

$$s(g) \notin \rho(T_i),$$

则:

$$\text{host}_i^T(g) = \perp.$$

而在 π^{bsp} 中, BSP 必须先成为合法证书:

$$\Gamma_i^T = \{g \in B_i : \text{host}_i^T(g) \neq \perp\}.$$

因此:

$$g \notin \Gamma_i^T.$$

所以它不能生成 voice。

于是得到:

$$g \in B_i, s(g) \notin \rho(T_i) \implies g \text{ 在 } \pi^{bsp, T} \text{ 中不可交易。}$$

如果子声部相关 BSP 大多在 orphan frontier, 那么:

$$\Gamma_i^{T, sub} \approx \emptyset,$$

从而:

$$A_i^{sub} = \emptyset.$$

这就是原架构下子声部结构性为零的严格原因。

2. 为什么不能用 workaround

三种常见 workaround 都是在偷换定义。

2.1 伪造父链

若:

$$\text{host}_i^T(g) = \perp$$

却人为设:

$$\text{host}(g) = c,$$

那就是放弃 P2。

2.2 用级别差当父子

若只因：

$$\ell(c) = \ell(g) + 1$$

就把 c 当作父，那就是放弃 P1 的 compose/subs 树关系。

2.3 放宽到区间包含

若改成：

$$\text{host}(g) = c \iff s(g) \in I(c),$$

那也是放弃 P2 的严格右端点命中。

所以，在 T_i 内坚持严格右端点命中时，orphan frontier BSP 必然无法交易。

3. 正确修法：区分结构树 T_i 和操作 carrier forest K_i

引入两个不同对象：

$$T_i = \text{结构视图树}$$

$$K_i = \text{操作 carrier forest.}$$

其中：

$$T_i \subseteq K_i \subseteq U_i \cup G_i.$$

这里 G_i 是必要的 germ / ambient carrier，用来承载尚未被高一级 compose 吸收的 frontier。

最简单方案是：

$$K_i = U_i$$

即操作层使用全量 tower 元素作为 carrier forest，而不是只使用最高 compose tree。

更严谨地，定义：

$$K_i = T_i \cup F_i^{op} \cup G_i,$$

其中 F_i^{op} 是有可能被 BSP host 的 frontier carrier， G_i 是必要的父级胚元容器。

要求 K_i 满足 endpoint-complete：

$$\forall g \in B_i, \quad \exists! c \in K_i \text{ such that } \rho(c) = s(g) \text{ and } c \text{ is the source carrier of } g.$$

然后定义新的 host：

$$\text{host}_i^K(g) = c \iff c \in K_i \wedge \rho(c) = s(g).$$

这仍然是严格右端点命中。

变化只在于 host 的定义域从 T_i 变成了 K_i 。

所以这不是“放宽 hostOf 到区间包含”，而是：

保留严格 endpoint host，但扩大操作 host 的载体宇宙。

4. 新定义下 BSP 可作为操作证书

定义 BSP 证书：

$$\gamma_g = (\text{host}_i^K(g), g, \delta(g), I_g, t_i),$$

其中：

$$I_g \subseteq \{1, 2, 3\}$$

保存三类买卖点类别。

若 g 是买点：

$$\delta(g) = +1.$$

若 g 是卖点：

$$\delta(g) = -1.$$

合法证书集合为：

$$\Gamma_i^K = \{\gamma_g : g \in B_i, \text{host}_i^K(g) \neq \perp\}.$$

由 endpoint-complete：

$$\text{host}_i^K(g) \neq \perp \quad \forall g \in B_i.$$

所以：

$$\Gamma_i^K \cong B_i.$$

也就是说，全格 BSP 都能进入操作解释器。

5. position voice 的严格定义

每个开仓 voice 不是 BSP 叶子，而是 position instance：

$$v = (c, \gamma, \sigma, n),$$

其中：

$$c = \text{host}_i^K(g),$$

$$\gamma = \gamma_g,$$

$$\sigma = \delta(g),$$

n 是 generation。

完整 position id：

$$\text{posId}(v) = H(c, \gamma, \sigma, n).$$

这就是完整 §13。

简化 §14:

$$\text{posId}^{simp}(v) = c$$

会合并 entry signal、方向和 generation，因此容易丢掉多实例语义。

但无论 §13 还是 §14，关键都不是“叶子开仓”，而是：

$$\boxed{BSP\ g\ \text{触发的是 carrier } c\ \text{上的 position voice } (c, \gamma, \sigma, n).}$$

6. 多重赋格父子关系

定义 carrier forest 的父关系：

$$\pi_K : K_i \rightarrow K_i \cup \{\perp\}.$$

若 $u = (c_u, \gamma_u, \sigma_u, n_u)$, $v = (c_v, \gamma_v, \sigma_v, n_v)$ ，则 voice 父关系为：

$$p(u) = v$$

当且仅当：

$$\pi_K(c_u) = c_v$$

并且 v 是该 carrier 上当前 active 的父声部实例。

ShortDiff 要求：

$$\boxed{\sigma_u = -\sigma_v.}$$

这就是：

- 父多，子空；
- 父空，子多。

7. AncOK 仍然是过滤器，不是生成器

raw voice set:

$$R_i = H_i^{live} \cup O_i,$$

其中：

$$O_i = \{v_\gamma : \gamma \in \Gamma_i^{acc}\}.$$

最终 active set:

$$\boxed{A_i = \text{AncOK}(R_i) = \{v \in R_i : \text{Anc}(v) \subseteq R_i\}.$$

所以：

$$A_i \subseteq R_i.$$

它不会自动添加父节点。

因此：

$$u \in A_i, d(u) > 0 \implies p(u) \in A_i.$$

换言之：

$$a_{u,i} \leq a_{p(u),i}.$$

这是约束，不是生成规则。

8. 如何让子声部可激活

子声部 u 要 active，必须满足两个条件：

$$u \in R_i,$$

以及：

$$\text{Anc}(u) \subseteq R_i.$$

也就是：

子声部自己的 BSP 证书被接受，且父声部已经 live 或同 bar 被 accepted certificate 打开。

这意味着不能只说：

$$g_u \in B_i.$$

必须有：

$$\gamma_u \in \Gamma_i^{\text{acc}}.$$

并且父声部 $v = p(u)$ 要么：

$$v \in H_i^{\text{live}},$$

要么：

$$v \in O_i.$$

即同 bar 内父、子同时被 accepted，也可以通过 AncOK，因为二者都在 raw set R_i 中。

9. 关键修复：解释器必须生成父 voice，而不是靠子 voice 反推父

定义解释器：

$$\mathcal{I}_\Theta(A_{i-1}, \Gamma_i^K) = (D_i, O_i, L_i).$$

其中：

- D_i ：要关闭的 voices；
- O_i ：要新开的 voices；
- L_i ：记录但不执行的证书。

若某个 BSP g 是某 carrier c 的入场证书，则：

$$\gamma_g \in \Gamma_i^K,$$

解释器可以创建：

$$v_g = (c, \gamma_g, \delta(g), n).$$

这时：

$$v_g \in O_i.$$

如果低一级子证书 g_u 也被接受，且其父 carrier 的 voice v_g 也在 O_i 或 H_i^{live} ，则：

$$u \in O_i, \quad p(u) = v_g,$$

于是：

$$\text{Anc}(u) \subseteq R_i,$$

所以：

$$u \in A_i.$$

这不是子声部强制生成父声部，而是解释器同时接受父、子两个证书。

10. 子声部激活定理

定理 1： 在 endpoint-complete 操作 carrier forest 下，子声部不再因 orphan host 结构性为零

假设：

1. K_i endpoint-complete；
2. hostOf 在 K_i 中严格右端点命中；
3. 每个 BSP 证书 g 生成 $\gamma_g \in \Gamma_i^K$ ；
4. 解释器 $\mathcal{I}_\Theta$ 接受一条父子证书链：

$$\gamma_0, \gamma_1, \dots, \gamma_d,$$

其 carriers 满足：

$$\pi_K(c_j) = c_{j-1},$$

方向满足：

$$\sigma_j = -\sigma_{j-1}$$

对 ShortDiff 层成立；

5. 对每个 γ_j ，生成 voice：

$$v_j = (c_j, \gamma_j, \sigma_j, n_j) \in O_i.$$

则：

$$v_d \in A_i.$$

证明

由 5：

$$v_j \in O_i \subseteq R_i \quad \forall j = 0, \dots, d.$$

由父子关系：

$$p(v_j) = v_{j-1}.$$

所以：

$$\text{Anc}(v_d) = \{v_{d-1}, \dots, v_0\}.$$

而这些全部属于 R_i 。

因此：

$$\text{Anc}(v_d) \subseteq R_i.$$

又：

$$v_d \in R_i.$$

所以：

$$v_d \in \text{AncOK}(R_i) = A_i.$$

证毕。

这说明，只要父子证书链真的进入 raw set，AncOK 不会剪掉子声部。

11. 原架构为什么失败，新架构为什么解决

原架构：

$$\Gamma_i^T = \{g \in B_i : s(g) \in \rho(T_i)\}.$$

若 BSP 大量在 orphan frontier，则：

$$\Gamma_i^{T,sub} \approx \emptyset.$$

所以：

$$O_i^{sub} \approx \emptyset.$$

因此：

$$A_i^{sub} = \emptyset.$$

新架构：

$$\Gamma_i^K = \{g \in B_i : s(g) \in \rho(K_i)\}.$$

由 endpoint-complete：

$$s(g) \in \rho(K_i) \quad \forall g \in B_i.$$

所以：

$$\Gamma_i^K \cong B_i.$$

如果解释器接受子声部证书且父证书链存在，则子声部可进 A_i 。

因此结构性为零被消除。

12. 这是否打破 P1?

严格说：

不打破 T_i 作为结构视图的 P1。

因为：

$$T_i = \downarrow r_i$$

仍然保持不变。

但它打破了“操作 host 只能来自 T_i ”这个额外约束。

所以必须把 spec 改成：

$$T_i = \text{structural view tree}, \quad K_i = \text{operational carrier forest.}$$

然后：

$$\text{host}^{struct} : B_i \rightarrow T_i$$

用于结构可视化或 bit-exact extract；

$$\text{host}^{op} : B_i \rightarrow K_i$$

用于交易解释器。

这两者不再混用。

如果你坚持：

$$\text{host}^{op} = \text{host}^{struct}$$

且 P1/P2 不变，那么无解，只能接受 C。

13. 这是否打破 P2？

如果 P2 的本质是：

$$\text{hostOf 必须严格右端点命中}$$

那不打破。

因为新 host 仍然满足：

$$\text{host}^K(g) = c \iff c \in K_i \wedge \rho(c) = s(g).$$

如果 P2 被定义成：

$$\text{hostOf 必须严格右端点命中 } T_i \text{ 内元素}$$

那就必须修改 P2。

所以最精确的修复是把 P2 拆成两条：

$$P2a : \text{strict endpoint hit}$$

$$P2b : \text{host universe}$$

保留 P2a，修改 P2b：

$$T_i \longrightarrow K_i.$$

这不是放宽到区间包含，也不是伪造父链。

14. 三类买卖点如何保留

每个 BSP 证书保留类别集合：

$$I_\gamma \subseteq \{1, 2, 3\}.$$

如果同一时刻同时有二类、三类买点：

$$I_\gamma = \{2, 3\}.$$

解释器使用一个全定义函数：

$$\text{priority}_\Theta(I_\gamma, c, \sigma, A_i)$$

和：

$$\text{size}_\Theta(I_\gamma, c, \sigma, A_i)$$

决定：

- 是否接受；
- 开多还是开空；
- 是开父声部还是开 ShortDiff 子声部；
- 仓位大小；
- 是否覆盖已有 voice；
- 是否只是记录。

因此三类买卖点不是丢掉，而是进入证书参数。

15. 全互斥解释器

候选证书集合有限：

$$\Gamma_i^K.$$

定义原始谓词：

$$P_1, \dots, P_m.$$

例如：

- P_1 : 风险强平,
- P_2 : 已有反向 voice 需要关闭,
- P_3 : 同 carrier 反手,
- P_4 : 父 voice active 且证书反父方向, 开 ShortDiff,
- P_5 : 无 active parent, 开 ambient voice,
- P_6 : 同向加仓或记录,
- P_7 : 不可执行, 记录.

互斥化：

$$\begin{aligned} C_1 &= P_1, \\ C_j &= P_j \wedge \bigwedge_{k < j} \neg P_k, \quad j = 2, \dots, m, \\ C_0 &= \bigwedge_{j=1}^m \neg P_j. \end{aligned}$$

则：

$$\sum_{j=0}^m \mathbf{1}[C_j] = 1.$$

证明：若无 P_j 成立，则 C_0 唯一成立；若有，取最小成立的 j ，则 C_j 唯一成立。

所以解释器是全互斥、全定义的。

16. 新策略全定义证明

定义完整状态：

$$x_i = (T_i, K_i, B_i, \Gamma_i^K, A_{i-1}, H_i, p_i, \dots).$$

策略步骤：

$$K_i = \text{OpCarrierForest}_{\Theta}(U_i, T_i),$$

$$\Gamma_i^K = \text{Cert}_{\Theta}(B_i, K_i),$$

$$(D_i, O_i, L_i) = \mathcal{I}_{\Theta}(A_{i-1}, \Gamma_i^K),$$

$$A_i = \text{AncOK}((A_{i-1} \setminus D_i) \cup O_i),$$

$$\tilde{p}_i = \text{LegTarget}(A_i),$$

$$p_i^* = \text{LexArgmin}_{p \in \mathcal{K}_{\Theta}(x_i)} J_i(p, \tilde{p}_i),$$

$$O_i^{order} = \text{Schedule}_{\Theta}(p_i^* - p_{i-1}).$$

于是：

$$\pi_{\Theta}^{bsp, K}(x_i) = \text{Schedule}_{\Theta} \left[\text{LexArgmin}_{p \in \mathcal{K}_{\Theta}(x_i)} J_i(p, \tilde{p}_i) - p_{i-1} \right].$$

若：

1. K_i 构造函数全定义；
2. hostOf^K 唯一；
3. Γ_i^K 有限；
4. 解释器 deterministic；
5. AncOK 是函数；
6. LegTarget 是函数；
7. 可行集非空有限；
8. LexArgmin 有固定平局；
9. Schedule 是函数；

则：

$$\forall x_i, \quad \exists! O_i^{order} = \pi_{\Theta}^{bsp, K}(x_i).$$

证明逐步由函数复合和有限最小值存在性得到。

17. 与原 $\Delta \text{Sharpe}=0$ 的关系

原来的零贡献定理是：

$$\Gamma_i^T \text{ 太小} \Rightarrow A_i^{sub} = \emptyset \Rightarrow \Delta N_i = 0 \Rightarrow \Delta \text{Sharpe} = 0.$$

新方案消除的是第一步：

$$\Gamma_i^K \cong B_i.$$

所以不能再由 host domain 推出：

$$A_i^{sub} = \emptyset.$$

但这只说明“可以被交易到”，不保证有 alpha。

新的判定要看：

$$\Delta N_i = N_i^{bsp,K} - N_i^{base}$$

是否非零，以及：

$$\Delta N_i \Delta P_{i+1}$$

是否有正的风险调整收益。

18. 如果选择 C，则无需修

如果你坚持：

$$T_i = \downarrow r_i$$

并且：

$$\text{host}(g) \in T_i$$

且必须严格右端点命中，那么数学上唯一自洽结论是：

$$\pi^{bsp} \text{ 只交易 hostable BSP。}$$

orphan frontier 上的 BSP 不属于当前策略定义域。

那就应把 goal 改成：

$$\text{测试最高 compose tree hostable BSP 是否有 alpha。}$$

而不是：

$$\text{测试全格 BSP 是否有 alpha。}$$

如果目标是全格 BSP，那么必须采用 K_i 。

19. 最终建议

最严格、最少破坏的解决方案是：

$$\text{双视图架构：} T_i \text{ 保持 P1，} K_i \text{ 用于操作 host。}$$

具体改法：

$$\begin{aligned}
T_i &= \text{ExtractTree}(D_i) = \downarrow r_i, \\
K_i &= \text{OpCarrierForest}(D_i) = T_i \cup F_i^{op} \cup G_i, \\
\text{host}^{struct} : B_i &\rightarrow T_i, \\
\text{host}^{op} : B_i &\rightarrow K_i, \\
\text{host}^{op}(g) = c &\iff c \in K_i \wedge \rho(c) = s(g).
\end{aligned}$$

然后 π^{bsp} 只使用：

$$\text{host}^{op}.$$

结构可视化、bit-exact extract、compose chain 仍使用：

$$\text{host}^{struct}$$

和：

$$T_i.$$

这解决了：

P1/P2 导致 orphan BSP 不可交易的问题。

同时没有使用：

- 伪造父链；
- 级别差强造父子；
- 区间包含 host。

最终严格结论：

若要全格 BSP 参与多重赋格子声部，必须把操作 host 宇宙从 T_i 扩展到 endpoint-complete

否则，保留原 $P1 \wedge P2$ 时，子声部结构性为零是定理，不是工程 bug。



